

VIETNAM NATIONAL UNIVERSITY - HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



DATA STRUCTURES AND ALGORITHMS - CO2003

ASSIGNMENT 2

**OPTIMIZING THE BotKify APPLICATION
USING AVL TREE DATA STRUCTURE**

Ho Chi Minh City, 03/2026

ASSIGNMENT'S SPECIFICATION

Version 1.0

1 Assignment's outcome

After completing this major assignment, students will be able to:

- Apply object-oriented programming (OOP) principles proficiently through advanced class design, inheritance, and polymorphism.
- Understand, design, and implement self-balancing tree data structures, specifically the AVL Tree and its Threaded variant.
- Apply advanced tree-based data structures to upgrade and optimize a real-world system (Playlist management for the BotKify application), significantly reducing computational complexity.

2 Introduction

In Assignment 1, students successfully implemented the foundational features of a music streaming application named BotKify. The system supported adding, removing, and querying songs within a playlist based on list data structures. While effective for small datasets, linear structures reveal severe limitations in computational complexity as the number of songs grows, typically requiring $O(n)$ time for search, insertion, and deletion operations.

In this assignment, students are required to upgrade the BotKify application to handle large-scale datasets efficiently. To achieve this, the linear lists will be replaced with a more advanced data structure: a key-value **AVL Tree**. The upgraded application will manage playlist data while strictly ensuring that insertion, deletion, and search operations are optimized to strictly $O(\log n)$ time complexity.

The upgraded BotKify system must support the following core functionalities:

- **Song Management:** Insert, delete, and retrieve songs in the playlist. Songs are stored in a sorted order (alphabetically by song title, with the song ID acting as a tie-breaker) using the AVL Tree.
- **Continuous Playback:** Users can smoothly transition to the next or previous song in the playlist based on the sorted alphabetical order.

- **Advanced Operations:** Calculating total playlist scores, comparing different playlists, playing random tracks, and approximate playback navigation.

In a standard AVL tree, finding the in-order predecessor or successor for the continuous playback feature takes $O(\log n)$ time. Students are highly encouraged to extend their implementation into a **Threaded AVL Tree** — a variant where `next` and `prev` pointers are added to each node to connect them into a doubly linked list following the In-order traversal. This structure allows the application to retrieve the next or previous song in strictly $O(1)$ time complexity.

3 AVL Tree and Threaded AVL Tree

Before introducing the AVL Tree, let's review the binary search tree (BST). A binary search tree (BST) is a hierarchical data structure consisting of nodes, where each node has at most two children, called the left child and the right child. In a BST, the left child of a node contains a value less than the node's value, and the right child contains a value greater than or equal to the node's value.

While BST makes search, insertion, and deletion operations efficient with a time complexity of $O(\log n)$ for balanced trees, it can degrade to $O(n)$ if the inserted data is already sorted, resulting in a skewed tree. To guarantee $O(\log n)$ operations in all cases, we use a self-balancing BST called the AVL Tree.

3.1 AVL Tree

An AVL tree is a self-balancing binary search tree. In an AVL tree, the height difference between the two child subtrees (left and right) of any node must not exceed one. If at any time this difference becomes greater than one, the tree automatically performs a rebalancing process to restore this property.

To maintain this strict balanced state, whenever an insert or erase operation disrupts the structure, the AVL tree utilizes basic rotations such as left rotation, right rotation, or double rotations. Thanks to this self-adjusting mechanism, all search, insertion, and deletion operations on the AVL tree are guaranteed to achieve an optimal time complexity of $O(\log n)$ in both average and worst-case scenarios.

Students are advised to consult the lecture materials to implement the AVL tree.

3.2 Threaded AVL Tree

Before introducing the Threaded AVL Tree, let's review the standard AVL Tree traversal. As presented in the previous sections, an AVL Tree ensures that search, insertion, and deletion operations maintain a time complexity of $O(\log n)$. However, when we need to find the in-order predecessor or successor of a node—a fundamental operation for continuous playback in a playlist—we must traverse up and down the tree branches. In the worst case, this traversal also takes $O(\log n)$ time.

The "Thread" in a Threaded AVL Tree represents a direct link between a node and its in-order predecessor or successor. While a standard AVL tree node contains only left and right child pointers to represent the hierarchical structure, a Threaded AVL Tree node contains two additional pointers: **prev** and **next**. These pointers are designed to handle sequential data traversal efficiently.

In a Threaded AVL tree, each node represents a track in the playlist. Like a standard AVL Tree, each node has a hierarchical position determined by its key (e.g., the song's title). But additionally, the **prev** pointer links directly to the node that comes immediately before it in alphabetical order, and the **next** pointer links directly to the node that comes immediately after it. This creates a doubly linked list embedded directly within the tree structure.

To facilitate understanding, let's refer to the following example. Let's insert all the listed song IDs into a Threaded AVL Tree in this sequence:

50, 30, 70, 20, 40

The root node is 50. Since it is the only node, its **prev** and **next** pointers are both NULL. It acts as both the head and the tail of the threaded list.

When we insert 30, it becomes the left child of 50. In an in-order traversal, 30 comes before 50. Therefore, 30's **next** pointer points to 50, and 50's **prev** pointer points back to 30. Node 30 becomes the new head of the threaded list.

Now when we insert 70, it becomes the right child of 50. In an in-order sequence, 70 comes after 50. Therefore, 50's **next** pointer points to 70, and 70's **prev** pointer points to 50. Node 70 becomes the new tail.

Continuing this process with 20 and 40, we eventually get the final tree, where the hierarchical edges maintain the BST property for $O(\log n)$ search, and the thread edges maintain a perfect linear sequence: 20 $\leftrightarrow$ 30 $\leftrightarrow$ 40 $\leftrightarrow$ 50 $\leftrightarrow$ 70.

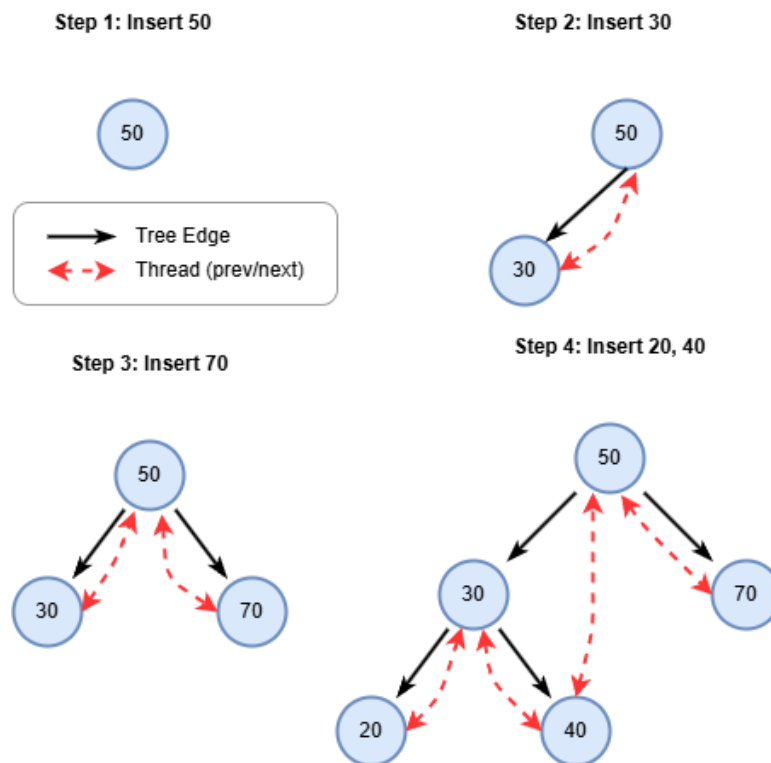


Figure 1: Step-by-step insertion of Threaded AVL tree

3.2.1 Insertion Algorithm

Insertion in a Threaded AVL tree operates exactly like a standard AVL tree, with the adjustment that after the hierarchical insertion, we must wire the new node into the doubly linked list.

The algorithm for maintaining threads during insertion proceeds as follows:

1. Traverse down the tree to find the correct insertion point as usual.
2. During the traversal, keep track of the logical in-order predecessor (**pred**) and in-order successor (**succ**) of the position where the new node will be placed.
3. Create the new node.
4. Wire the threads: Set the new node's **prev** to **pred**, and its **next** to **succ**.
5. Update the adjacent nodes.
6. Trace back to the root, update balance factors, and perform AVL rotations if necessary.

Example 3.1

Let's continue from the previous Threaded AVL tree which has the threaded sequence: 20 <-> 30 <-> 40 <-> 50 <-> 70. The root of the tree is 50.

Insert 35

- **Insertion & Wiring:** 35 is physically inserted as the left child of 40. We logically wire it between its predecessor (30) and successor (40) in the threaded list: 30 $\leftrightarrow$ 35 $\leftrightarrow$ 40.
- **AVL Rotation:** The insertion causes an imbalance at the root 50 (LR case). A **Left-Right Rotation** is performed, making 40 the new root.

Insert 80

- **Insertion & Wiring:** From the new root 40, 80 is physically inserted as the right child of 70. We append it to the end of the threaded list: 70 $\leftrightarrow$ 80.
- **AVL Rotation:** Node 50 becomes unbalanced (RR case). A **Left Rotation** is performed, pulling 70 up to become the new parent of 50.

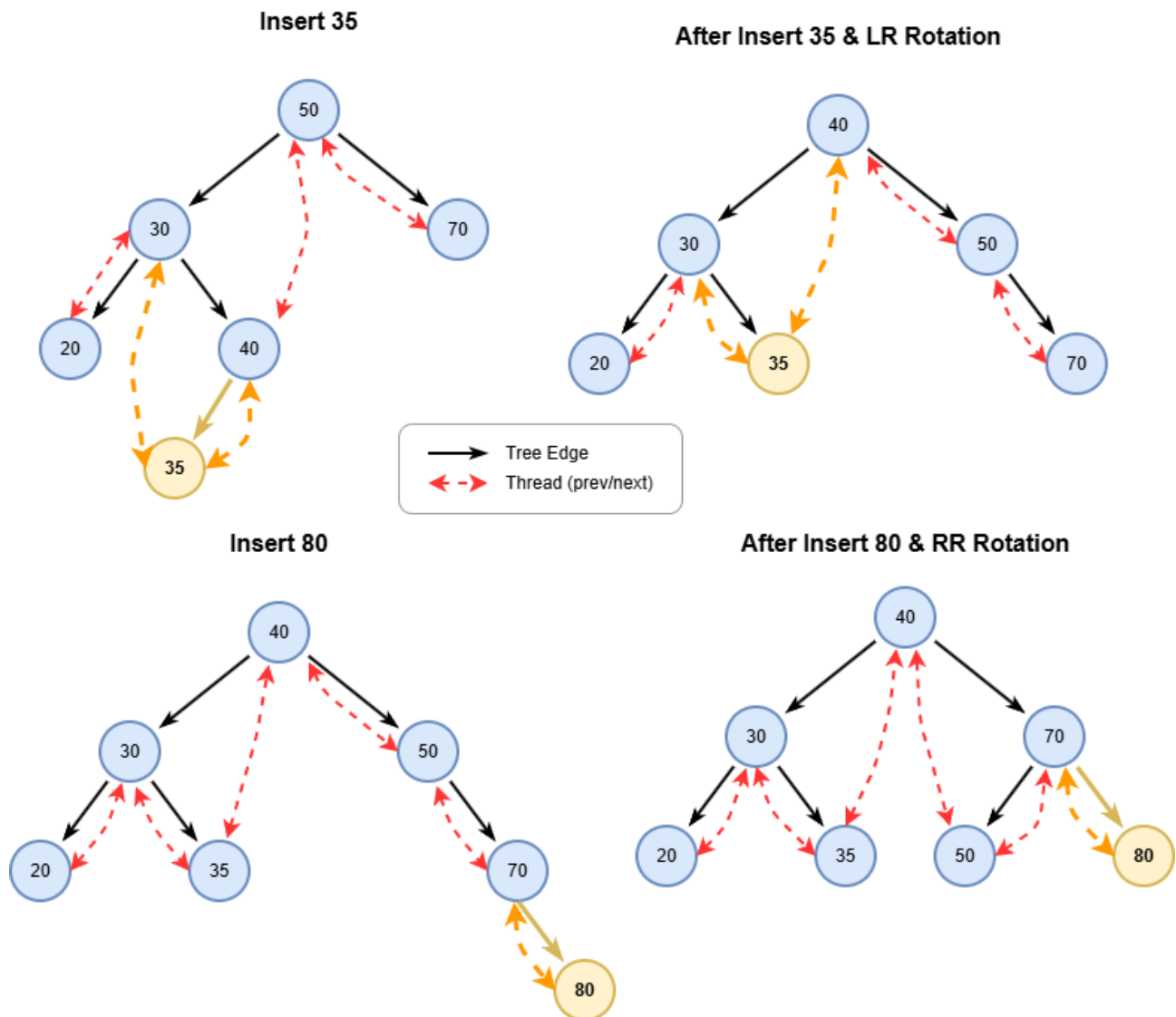


Figure 2: Step-by-step insertion of 35 and 80 to Threaded AVL tree

3.2.2 Deletion Algorithm

Deletion in a Threaded AVL Tree requires precise identification of the **physical node** that is being removed from memory, as this is the node that must be bypassed in the doubly linked list.

Our deletion process will work as follows. This process is recursive, assuming the node to be deleted is u :

- (a) **If u has 0 or 1 child:** The node u itself will be physically deleted from the tree. Before deleting u , we must bypass it in the threaded list. Let P be u 's **prev** and S be u 's **next**.

We link P and S directly to each other ($P \leftrightarrow S$), completely bypassing u . Then, we proceed with standard AVL deletion and balancing.

- (b) **If u has 2 children:** We cannot physically delete u . Instead, we find its in-order successor r (the smallest node in its right subtree). We copy the data (the song information) from r into u . At this point, the node u effectively becomes r in terms of its data. The physical node that will actually be deleted from the tree memory is r . Therefore, we must rewire the threads to bypass the physical node r , connecting r 's **prev** directly to r 's **next**. After rewiring, we recursively delete the old physical node r .

Example 3.2

Let's consider a Threaded AVL Tree with the sequence: 20 $\leftrightarrow$ 30 $\leftrightarrow$ 40 $\leftrightarrow$ 50 $\leftrightarrow$ 60 $\leftrightarrow$ 70 $\leftrightarrow$ 80 $\leftrightarrow$ 90. The root of the tree is 50.

- **Initial State:** Node 50's left child is 30. Its right child is 70. Node 70 has a left child (60) and a right child (80).
- **Target:** We want to delete the root node 50 (which has 2 children).

Step 1: Find Successor and Copy Data

1. The in-order successor of 50 is 60 (the smallest node in 50's right subtree).
2. We copy the data '60' into the physical root node. The physical root node is no longer 50; it now holds 60.
3. The original, physical node 60 (which is the left child of 70) is the one that will be permanently deleted from the tree's memory.

Step 2: Rewire Threads (Bypass the deleted physical node)

1. Before physically freeing node 60, we must bypass it in the doubly linked list.
2. The predecessor of the physical node 60 is the physical root node (which now holds '60'). Its successor is 70.
3. We link them directly: $\text{Root} \rightarrow \text{next} = 70$ and $70 \rightarrow \text{prev} = \text{Root}$. The sequence perfectly updates to ... $\leftrightarrow$ 40 $\leftrightarrow$ 60 $\leftrightarrow$ 70 $\leftrightarrow$ 80 $\leftrightarrow$ 90.

Step 3: AVL Balancing and Rotation

1. We physically delete the original node 60.
2. We trace back up to update balance factors. Node 70 has lost its left child. Its left subtree height is now 0, but its right subtree (rooted at 80) has height 2.
3. **Imbalance!** Node 70's balance factor becomes RR Case.
4. A **Left Rotation** is performed at node 70. Node 80 moves up to become the new right child of the root. Node 70 moves down to become the left child of 80.

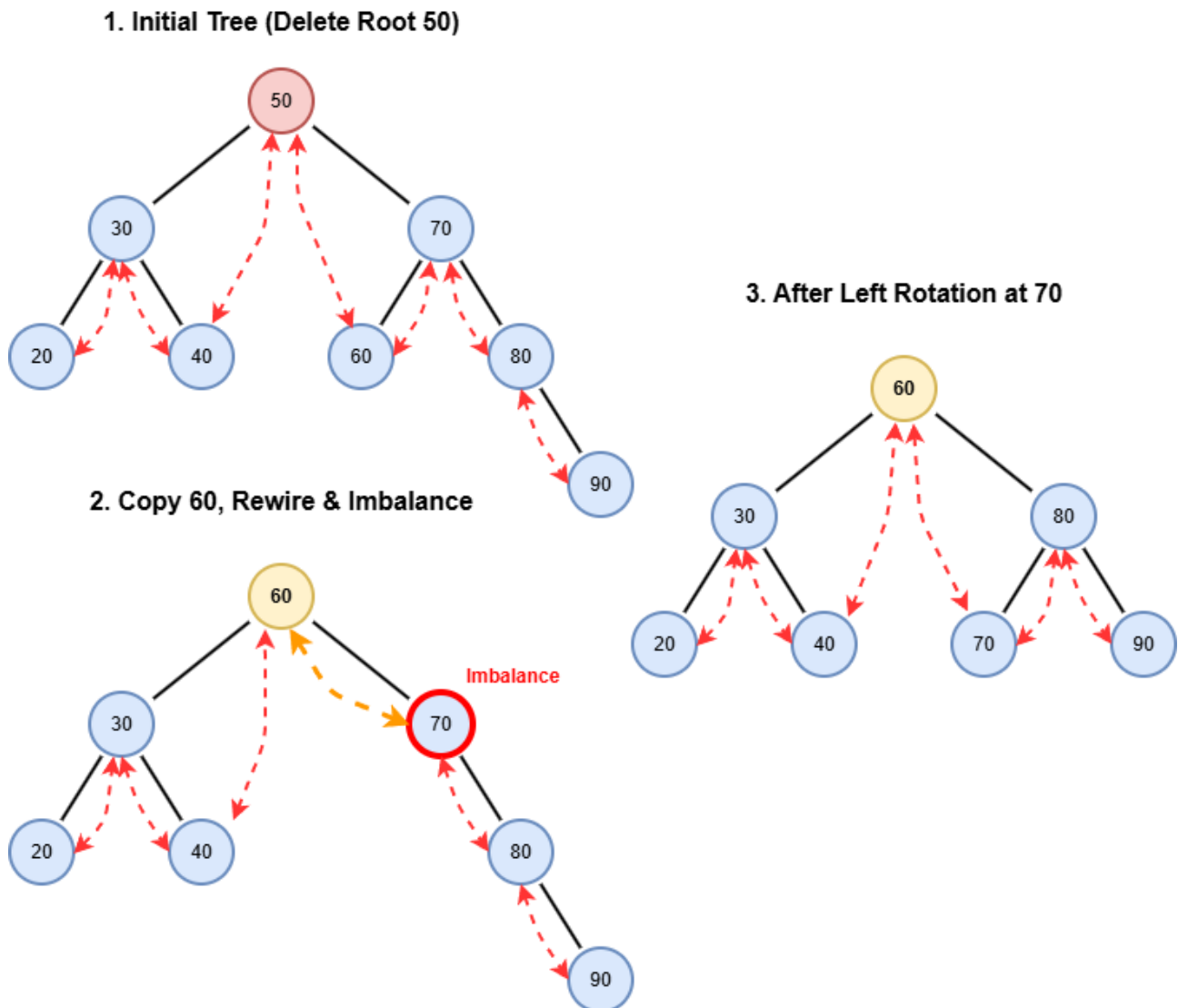


Figure 3: Step-by-step deletion in Threaded AVL tree

4 Requirements

In this assignment, students are required to implement the following classes to describe the data processing process and implement the music streaming application using the AVL Tree structure, including the IBST interface, AVL class, ThreadedAVL class, Song class, and Playlist class.

4.1 Interface IBST

The IBST class is an abstract interface that defines the standard operations for a key-value binary search tree.

The description for the IBST class is as follows:

```
1 template <typename K, typename V>
2 class IBST {
3 public:
4     virtual ~IBST() = default;
5     virtual bool insert(const K& key, const V& value) = 0;
6     virtual bool erase(const K& key) = 0;
7     virtual V* find(const K& key) = 0;
8     virtual bool contains(const K& key) const = 0;
9     virtual int size() const = 0;
10    virtual bool empty() const = 0;
11    virtual void clear() = 0;
12    virtual int height() const = 0;
13    virtual std::list<K> ascendingList() = 0;
14    virtual std::list<K> descendingList() = 0;
15    virtual std::string toString(
16        std::string (*entry2str)(const K&, const V&) = nullptr
17    ) const = 0;
18 };
```

Because this is a pure abstract class, students do not need to implement these methods directly in IBST, but must override and implement all of them in the derived AVL class.

4.2 Class AVL (4 points)

The AVL class inherits from IBST and stores data in a self-balancing AVL tree structure. It provides users with balanced operations ensuring $O(\log n)$ time complexity.

The description for the AVL class is as follows:

```
1 template <typename K, typename V>
2 class AVL : public IBST<K, V> {
3 public:
4     struct Node {
5         K key;
6         V value;
7         int bfactor;
8         Node* pLeft;
9         Node* pRight;
10        Node(const K& k, const V& v);
11        virtual ~Node() = default;
12        std::string toString(...) const;
13    };
```

```
14 protected:
15     Node* pRoot;
16     virtual void onInserted(Node* newNode, Node* pred, Node* succ) {}
17     virtual void onErasing(Node* delNode, Node* pred, Node* succ) {}
18     virtual void onReplaceBySuccessor(Node* target, Node* succ, Node*
succNext) {}
19     virtual Node* createNode(const K& key, const V& value) {
20         return new Node(key, value);
21     }
22 public:
23     AVL();
24     virtual ~AVL();
25     // Inherited methods from IBST...
26 };
```

Below is detailed information about the methods and attributes in the AVL class:

1. `Node* pRoot;`
 - Root node of the AVL tree. Initialized to `nullptr`.
2. `virtual Node* createNode(const K& key, const V& value);`
 - Factory method to create a new `Node`. Returns a pointer to the newly allocated node.
3. `virtual void onInserted(Node* newNode, Node* pred, Node* succ);`
 - Hook method called immediately after `newNode` is inserted into the tree.
 - `pred` and `succ` are the in-order predecessor and successor of `newNode`, respectively.
 - In the AVL class, this method does nothing. It is designed to be overridden by `ThreadedAVL`.
4. `virtual void onErasing(Node* delNode, Node* pred, Node* succ);`
 - Hook method called right before a node with 0 or 1 child (`delNode`) is physically deleted.
 - In the AVL class, this method does nothing.
5. `virtual void onReplaceBySuccessor(Node* target, Node* succ, Node* succNext);`
 - Hook method called during the deletion of a `target` node that has 2 children. `succ` is the in-order successor of the target, and `succNext` is the node that comes immediately after `succ`.
 - In the AVL class, this method does nothing.

6. `bool insert(const K& key, const V& value);`

- Inserts a new key-value pair into the tree.
- If the `key` already exists, the function does not add it and returns `false`.
- After insertion, the tree must update balance factors and perform rotations (LL, RR, LR, RL) if necessary to maintain the AVL property.
- **Hint:** Students must properly determine `pred` and `succ` and invoke `onInserted` before returning `true`.

7. `bool erase(const K& key);`

- Removes the node containing the given `key`.
- If the key is not found, returns `false`.
- After deletion, the tree must trace back to the root, update balance factors, and perform rotations to maintain the AVL property.
- **Hint:** Must invoke `onErasing` or `onReplaceBySuccessor` correctly depending on the number of children the deleted node has.

8. `V* find(const K& key);`

- Returns a pointer to the value associated with the `key`. Returns `nullptr` if the key is not found. Time complexity must be $O(\log n)$.

9. `std::list<K> ascendingList();`

- Traverses the tree in In-order and returns a `std::list` containing all keys in ascending order.

10. `int height() const;`

- Returns the current height of the tree. If the tree has no elements, the height is 0.

4.3 Class ThreadedAVL (3 points)

The `ThreadedAVL` class extends the `AVL` class by adding threading capabilities (doubly linked list) to support $O(1)$ in-order traversal.

The description for the `ThreadedAVL` class is as follows:

```
1 template <typename K, typename V>
2 class ThreadedAVL : public AVL<K, V> {
3 private:
4     using Base = AVL<K, V>;
5     using AVLNode = typename Base::Node;
6     struct TNode : public Base::Node {
```

```
7     TNode* prev;
8     TNode* next;
9     TNode(const K& k, const V& v);
10 };
11 TNode* head_;
12 TNode* tail_;
13 public:
14     class Iterator {
15     public:
16         Iterator& operator++();
17         Iterator& operator--();
18         const K& key() const;
19         V& value();
20         bool isNull() const;
21     };
22     ThreadedAVL();
23     ~ThreadedAVL() override;
24
25     std::list<K> ascendingList() override;
26     std::list<K> descendingList() override;
27     void clear() override;
28
29     Iterator beginIt();
30     Iterator endIt();
31     Iterator rbeginIt();
32     Iterator findIt(const K& key);
33
34 protected:
35     AVLNode* createNode(const K& key, const V& value) override;
36     void onInserted(AVLNode* newNode, AVLNode* pred, AVLNode* succ) override
37     ;
38     void onErasing(AVLNode* delNode, AVLNode* pred, AVLNode* succ) override;
39     void onReplaceBySuccessor(AVLNode* target, AVLNode* succ, AVLNode*
succNext) override;
40 };
```

Below is detailed information about the methods in the ThreadedAVL class:

1. TNode* head_; and TNode* tail_;
 - Pointers to the smallest and largest nodes in the tree, representing the head and tail of the doubly linked list.
2. AVLNode* createNode(const K& key, const V& value);

- Overrides the base class factory method to dynamically allocate and return a `TNode` instead of a standard `Node`.
- 3. `void onInserted(...);, void onErasing(...);, void onReplaceBySuccessor(...);`
 - Students must implement the logic to correctly rewire the `prev` and `next` pointers of the affected nodes to maintain the threaded sequence perfectly.
- 4. `std::list<K> ascendingList();`
 - Overrides the recursive traversal. This method must collect keys by iterating through the `next` pointers starting from `head_`.
- 5. `Iterator operator++();` and `Iterator operator--();`
 - Advances the iterator to the `next` or `prev` node in exactly $O(1)$ time.

4.4 Class Song

The `Song` class represents a single track in the playlist.

The description for the `Song` class is as follows:

```
1 class Song {
2 private:
3     int id;
4     std::string title;
5     std::string artist;
6     std::string album;
7     int duration;
8     int score;
9     std::string url;
10    int play_count;
11 public:
12    Song(int id, std::string title, std::string artist, std::string album,
13         int duration, int score, std::string url);
14    std::string toString() const;
15 };
```

4.5 Class Playlist (3 points)

The `Playlist` class is the core application module that manages a collection of songs using the underlying AVL Tree.

```
1 using SongKey = std::pair<std::string, int>; // (title, id)
2
3 class Playlist {
4 private:
5     std::string name;
6     int size;
7     int currentIndex;
8 #ifdef USE_THREADED_AVL
9     ThreadedAVL<SongKey, Song*> songs;
10    ThreadedAVL<SongKey, Song*>::Iterator currentIt;
11    bool hasCurrent;
12 #else
13    AVL<SongKey, Song*> songs;
14 #endif
15 private:
16    SongKey makeKey(Song* s) const;
17    void resetPlayback();
18 public:
19    Playlist(std::string name);
20    int getSize() const;
21    bool empty() const;
22    void clear();
23    void addSong(Song* s);
24    void removeSong(int index);
25    Song* getSong(int index) const;
26    Song* playNext();
27    Song* playPrevious();
28    int getTotalScore();
29    bool compareTo(const Playlist& p, int numSong);
30    void playRandom(int index);
31    int playApproximate(int step);
32 };
```

Below is detailed information about the data members and methods in the Playlist class:

1. `std::string name;`
 - The name of the playlist.
2. `int size;`
 - The current number of songs in the playlist. Students must update this variable whenever a song is added or removed.
3. `int currentIndex;`

- The 0-based index of the currently playing song according to the sorted in-order sequence.
- If no song has been played yet, or if the playlist is empty, this variable should be set to a default value (e.g., -1).

4. `AVL<SongKey, Song*> songs; (or ThreadedAVL)`

- The tree data structure used to store and manage the songs. The key is `SongKey` and the value is a `Song*` pointer.

5. `Iterator currentIt; and bool hasCurrent; (Only applicable with USE_THREADED_AVL)`

- `currentIt`: An Iterator pointing directly to the currently playing song's node, supporting $O(1)$ song transitions.
- `hasCurrent`: A flag indicating whether the current playback state is valid (helps to distinguish when the iterator has not been initialized or when the currently playing song has been removed).

6. `SongKey`

- Defined as `std::pair<std::string, int>`. Songs are ordered alphabetically by `title`. If two songs have the exact same title, the `id` is used as a tie-breaker to ensure uniqueness in the AVL tree.

7. `void addSong(Song* s);`

- Inserts the song pointer `s` into the AVL tree using the `SongKey`. Increments the `size`.

8. `void removeSong(int index);`

- Removes the song at the specified 0-based `index` according to the sorted in-order sequence.
- Decrements the `size`. If the currently playing song is removed, playback state must be properly reset.

9. `Song* getSong(int index) const;`

- Returns the pointer to the song at the specified 0-based `index` according to the sorted in-order sequence. Returns `nullptr` if the index is out of bounds.

10. `Song* playNext();`

- Advances the playback to the next song in the sorted sequence and returns its pointer.

- If no song is currently playing, it starts from the first song. If already at the last song, it returns `nullptr`.
- **Hint:** If compiled with `USE_THREADED_AVL`, this must utilize the Iterator's `++` operator to achieve $O(1)$ time complexity.

11. `Song* playPrevious();`

- Rewinds the playback to the previous song in the sorted sequence and returns its pointer. Returns `nullptr` if it is at the first song.

12. `int getTotalScore();`

- Returns the sum of the `score` attribute of all songs currently in the playlist.

13. `bool compareTo(const Playlist& p, int numSong);`

- Compares the current playlist with playlist `p` based on the total score of their first `numSong` songs (in sorted sequence).
- Returns `true` if the current playlist's calculated score is strictly greater than or equal to `p`'s score.

14. `void playRandom(int index);`

- Jumps the playback state directly to the song at the specified `index`.

15. `int playApproximate(int step);`

- From the current playback position, jumps forward (if `step > 0`) or backward (if `step < 0`) exactly `|step|` positions in the sorted sequence.
- Returns the new `currentIndex`.

5 Instructions

To complete this assignment, students must:

1. Read through this entire description file.
2. Download the initial.zip file and extract it. After extraction, students will receive the following files: `main.cpp`, `main.h`, `utils.h`, `IBST.h`, `AVL.h`, `ThreadedAVL.h`, `Playlist.h`, and `Playlist.cpp`. Students must only submit the `AVL.h`, `ThreadedAVL.h`, and `Playlist.cpp`, so they should not modify the `main.h`, `IBST.h`, and `Playlist.h` files when running the program.
3. Students should use the following command to compile:

Basic mode:

```
g++ -o main main.cpp Playlist.cpp -I . -std=c++17
```

Full mode:

```
g++ -o main main.cpp Playlist.cpp -I . -std=c++17  
-DUSE_THREADED_AVL
```

This command is used in the command prompt/terminal to compile the program. If students use an IDE to run the program, they need to note the following: add all necessary files to the IDE's project/workspace; change the IDE's compilation command accordingly. IDEs typically provide buttons for compiling (Build) and running the program (Run). When pressing Build, the IDE will execute a corresponding compilation command, usually only compiling main.cpp. Students need to configure the IDE to change the compilation command: add the Playlist.cpp file, add the options -std=c++17, -I ., and -DUSE_THREADED_AVL (if implementing the Threaded AVL part).

4. The program will be graded on the UNIX platform. The platform and compiler used by students may differ from the actual grading environment. The submission platform on BKeL is set up similarly to the actual grading environment. Students must run the program on the submission platform and fix any errors that occur on BKeL to ensure correct results during actual grading.
5. Modify the AVL.h, ThreadedAVL.h, and Playlist.cpp files to complete this assignment and ensure the following requirements are met:
 - All methods described in this specification must be implemented so that compilation is successful. If students cannot implement a method, provide an empty implementation for that method. Each testcase will call some of the described methods to check the return results.
 - There is only 1 **include** directive in the file AVL.h, which is **#include "IBST.h"**. There is only 1 **include** directive in the file ThreadedAVL.h, which is **#include "AVL.h"**. There is only 1 **include** directive in the file Playlist.cpp, which is **#include "Playlist.h"**. Additionally, no other **#include** directive is allowed in these files.
6. The main.cpp file provides some simple testcases in functions with the format **"tc<x>()"**.
7. Students are allowed to write additional methods and attributes within the required classes. Students are also permitted to write additional classes beyond the required implementations.
8. Students are required to design and use data structures based on the types of trees they have learned.
9. Students must deallocate all dynamically allocated memory when the program ends.

6 Submission

Students must only submit 3 files: AVL.h, ThreadedAVL.h, and Playlist.cpp, before the deadline specified in the "Assignment 2 - Submission" path. There are some simple testcases used to check the students' work to ensure that the results can be compiled and run. Students can submit their work as many times as they want, but only the last submission will be graded. Because the system cannot handle the load when too many students submit their work at the same time, students should submit their work as early as possible. Students will bear the risk if they submit their work close to the deadline. Once the deadline for submission has passed, the system will be closed, and students will not be able to submit anymore. Submissions through other methods will not be accepted.

7 Other regulations

- Students must complete this assignment on their own and must prevent others from stealing their results. Failure to do so will result in disciplinary action according to the university's cheating policy.
- Any decision made by the teachers in charge of this assignment is the final decision.
- Students are not provided with testcases after grading, but are only provided with information on testcase design strategies and distribution of the correct number of students according to each test case.
- Assignment contents will be **harmonized** with some questions in exam with similar content.

8 Changelog

- Initial release of Assignment 2 Specification for the current semester.

—————END—————